

Multi-Objective Optimization with JuMP

Julia, JuMP and MOA from a student perspective

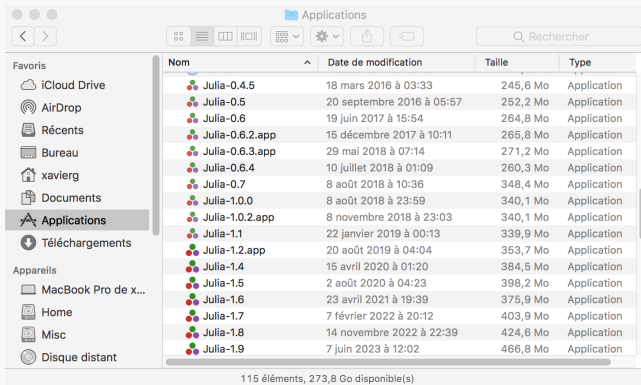
JuliaCon 2023 | JuMP-dev – Uses of JuMP in teaching

Xavier GANDIBLEUX

<https://xgandibleux.github.io/>

A chat at ROADEF in Feb 2016

Since 2017, Julia and JuMP...



...as support of exercises and projects in my OR courses.

Lecturing with Julia and JuMP

- Universities: year of study:
 - Nantes University (France) CS: 1 2 3 4 5
 - University of South Brittany (France) maths: 1 2 3 4 5
 - Johannes Kepler University Linz (Austria) business: 1 2 3 4 5
- Courses:
 - Operations Research 3
 - Scientific Development Project 3
 - Discrete and Combinatorial Optimization 4
 - Metaheuristics 4
 - MultiObjective Optimization 4 5
 - Programming for Business Tasks 4 5

Lecturing with Julia and JuMP (1st course)

- Universities: year of study:
 - Nantes University (France) CS: 1 2 3 4 5
 - University of South Brittany (France) maths: 1 2 3 4 5
 - Johannes Kepler University Linz (Austria) business: 1 2 3 4 5
- Courses:
 - Operation Research 3
 - lectures (10×1h20), exercises (16×1h20), **labs** (6×1h20 + project)
 - . introduction to Julia and JuMP
 - . models (explicit, implicit; dense, space arrays), resolution (LP, IP)
 - . project: TSP (Miller-Tucker-Zemlin, Dantzig-Fulkerson-Johnson...)
 - Julia v.1.8.3, JuMP v1.10.0, GLPK
 - February 2023 to April 2023

Short Survey on 3 aspects: student – Julia – JuMP (1/3)

- Student:
1. Which programming language(s) have you used regularly or frequently during this academic year?

java (100%), **julia (100%)**, Ocaml (100%), C++ (56%), Python (18%)

2. Which programming language(s) would you like to use regularly or frequently during the next academic year?

java (46%), C++ (41%), **julia (26%)**, C (10%), Python (10%)

3. On a scale from 0 (min) to 5 (max), do you like to code in C / C++ / Java / Python / Julia / R / Rust / Other ?

Java (140), **Julia (135)**, C++ (134), C (75), Python (61), Rust (7), R(1)

Short Survey on 3 aspects: student – Julia – JuMP (2/3)

Julia:

1. Thinking about the technical/characteristic aspects of Julia, what are the aspects that you like the most (why)?
 - language and datastructures: clear, simple, intuitive, readable 62%
 - types: clear, easy to deal with types 15%
 - coding: flexibility thanks to the syntax and datastructures 8%
2. Thinking about the technical/characteristic aspects of Julia, what are the aspects that you like the least (why)?
 - error management: message not informative enough, debugger unclear 15%
 - structuration of code: no semi-column, no curly bracket 10%
 - documentation: some functions hard to understand/use; too dense 5%
3. What kind of improvement to make to Julia (language, environment, package management...) would you like to express?
 - docs: more; less complex; more examples; better description methods 10%
 - error management: better indication; clarify 8%

Short Survey on 3 aspects: student – Julia – JuMP (2/3)

Julia:

1. Thinking about the technical/characteristic aspects of Julia, what are the aspects that you like the most (why)?
 - language and datastructures: clear, simple, intuitive, readable 62%
 - types: clear, easy to deal with types 15%
 - coding: flexibility thanks to the syntax and datastructures 8%
2. Thinking about the technical/characteristic aspects of Julia, what are the aspects that you like the least (why)?
 - error management: message not informative enough, debugger unclear 15%
 - structuration of code: no semi-column, no curly bracket 10%
 - documentation: some functions hard to understand/use; too dense 5%
3. What kind of improvement to make to Julia (language, environment, package management...) would you like to express?
 - docs: more; less complex; more examples; better description methods 10%
 - error management: better indication; clarify 8%

Short Survey on 3 aspects: student – Julia – JuMP (3/3)

JuMP:

1. What are the technical aspects of JuMP that you found most intuitive/facilitating to deal with OR problem?
 - very intuitive usage: @variable, @objective, @constraint, optimize! 69%
 - easy way to declare a problem, easy to use, same as on paper (natural) 36%
 - easy way to solve an OR problem 5%
2. What technical aspects of JuMP did you find unintuitive/rustic to deal with an OR problem?
 - extract data from a created model is not intuitive 5%
 - error messages 5%
 - displaying the statuses 5%
3. Based on your experience, which aspects of JuMP would you propose to revise/evolve and why?
 - heavy package, compilation too slow, precompile the package 15%
 - error messages sometime unclear (err msg is not helpful) 13%
 - reference/access to variables outside the method where the model is 5%

Short Survey on 3 aspects: student – Julia – JuMP (3/3)

JuMP:

1. What are the technical aspects of JuMP that you found most intuitive/facilitating to deal with OR problem?
 - very intuitive usage: @variable, @objective, @constraint, optimize! 69%
 - easy way to declare a problem, easy to use, same as on paper (natural) 36%
 - easy way to solve an OR problem 5%
2. What technical aspects of JuMP did you find unintuitive/rustic to deal with an OR problem?
 - extract data from a created model is not intuitive 5%
 - error messages 5%
 - displaying the statuses 5%
3. Based on your experience, which aspects of JuMP would you propose to revise/evolve and why?
 - heavy package, compilation too slow, precompile the package 15%
 - error messages sometime unclear (err msg is not helpful) 13%
 - reference/access to variables outside the method where the model is 5%

Lecturing with Julia and JuMP (2nd course)

- Universities: year of study:
 - Nantes University (France) CS: 1 2 3 4 5
 - University of South Brittany (France) maths: 1 2 3 4 5
 - Johannes Kepler University Linz (Austria) business: 1 2 3 4 5
- Courses:
 - Multi-Objective Optimization (online) 4
 - 8 sessions (theory, exercises); **session 6** (2 weeks):
 - . Julia and JuMP (revisions)
 - . `vOptGeneric` → `MultiObjectiveAlgorithms`
 - . models (explicit, implicit), resolution (2IP: lex, ϵ -cst; 3IP: Kirlik-S)
 - Julia v.1.8.3, JuMP v1.10.0, MOA v1.0, GLPK
 - February 2023 to June 2023

Lecturing with Julia and JuMP (2nd course)

- Universities: year of study:
 - Nantes University (France) CS: 1 2 3 4 5
 - University of South Brittany (France) maths: 1 2 3 4 5
 - Johannes Kepler University Linz (Austria) business: 1 2 3 4 5
- Courses:
 - Multi-Objective Optimization (online) 4
 - 8 sessions (theory, exercises); **session 6** (2 weeks):
 - . Julia and JuMP (revisions)
 - . `vOptGeneric` → `MultiObjectiveAlgorithms`
 - . models (explicit, implicit), resolution (2IP: lex, ϵ -cst; 3IP: Kirlik-S)
 - Julia v.1.8.3, JuMP v1.10.0, MOA v1.0, GLPK
 - February 2023 to June 2023

Short Survey: student – Julia – JuMP+MOA (1/3)

- Student:
1. Which programming language(s) have you used regularly or frequently during this academic year?

Python (100%), julia (100%), Octave (14%)

2. Which programming language(s) would you like to use regularly or frequently during the next academic year?

Python (83%), julia (83%)

3. On a scale from 0 (min) to 5 (max), do you like to code in C / C++ / Java / Python / Julia / R / Rust / Other ?

Python (27), Julia (23), R(15), C++ (8), C (7), Octave (5)

Short Survey: student – Julia – JuMP+MOA (2/3)

Julia:

1. Thinking about the technical/characteristic aspects of Julia, what are the aspects that you like the most (why)?
 - speed, syntax, easy, intuitive, readable
 - close to mathematics, work with Jupyter, maths libraries
2. Thinking about the technical/characteristic aspects of Julia, what are the aspects that you like the least (why)?
 - error messages not informative enough
 - documentation; first run
3. Based on your experience, why would you tell to someone to code in Julia (why)?
 - libraries, performances, types
 - easy to learn; easy to use: from an draft project to a final code

« A student in science should code in Julia because there is not a lot of simple AND very effective/efficient languages for scientific modeling »

Short Survey: student – Julia – JuMP+MOA (3/3)

JuMP+MOA: 1. What are the technical aspects of JuMP that you found most intuitive/facilitating to deal with a MOO problem?

- very simple, intuitive, syntax close to maths,
- single interface to several solvers

« Anyone can understand how to code with JuMP. MOA is simple to handle »

2. What technical aspects of JuMP did you find unintuitive/rustic to deal with a MOO problem?

- nothing to report
- error messages

3. Based on your experience, which aspects of JuMP would you propose to revise/evolve and why?

- nothing to report
- more examples

« The MOO examples given on the JuMP site allow quite well to see how to use the MOA package. But I couldn't find any explicit documentation describing the different attributes of the MOA package. A comprehensive documentation is welcome »

Julia and Optimization Days 2023

4-6 October 2023. Paris, France.

<https://julia-users-paris.github.io/workshop/en/>

Programme committee:

Mathieu Besançon (Zuse Institute Berlin) – Vincent Duval (Inria Paris) – Xavier Gandibleux (Nantes Université)
Vincent Leclère (Ecole des Ponts ParisTech) – Antoine Levitt (Laboratoire de mathématique d'Orsay)
Pierre Navaro (Institut de recherche mathématiques de Rennes) – François Pacaud (Mines Paris-PSL)